

ECE 260C, Spring 2026

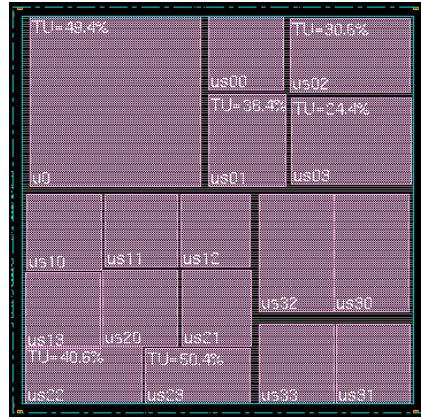
Routing

Andrew B. Kahng

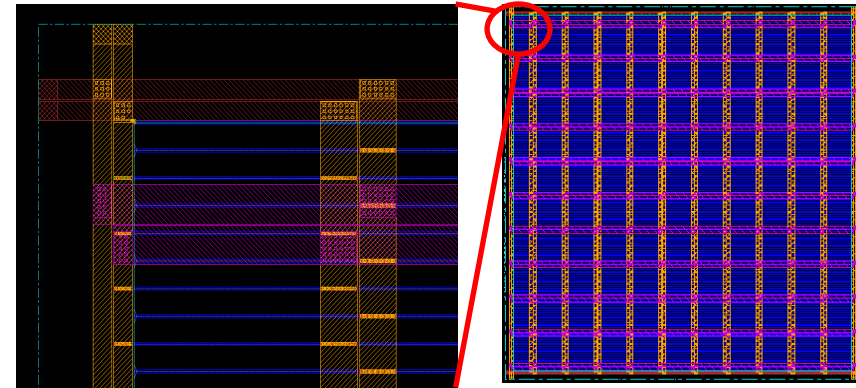
Thanks to: Bangqi Xu, Matt Liberty, Cho Moon, Eder Monteiro, Zhiang Wang, ...

Physical Design Flow Pictures (old ECE 260B slide)

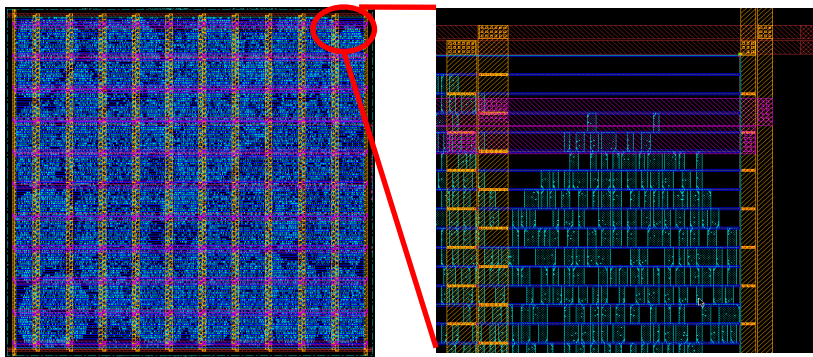
- Floorplanning



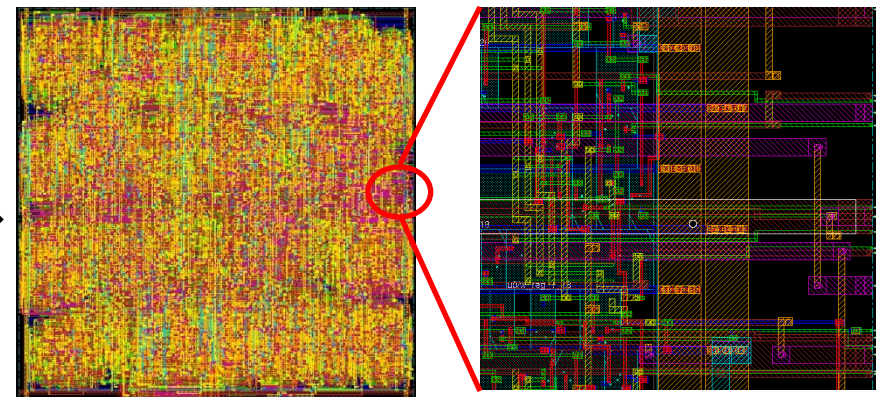
- Powerplanning



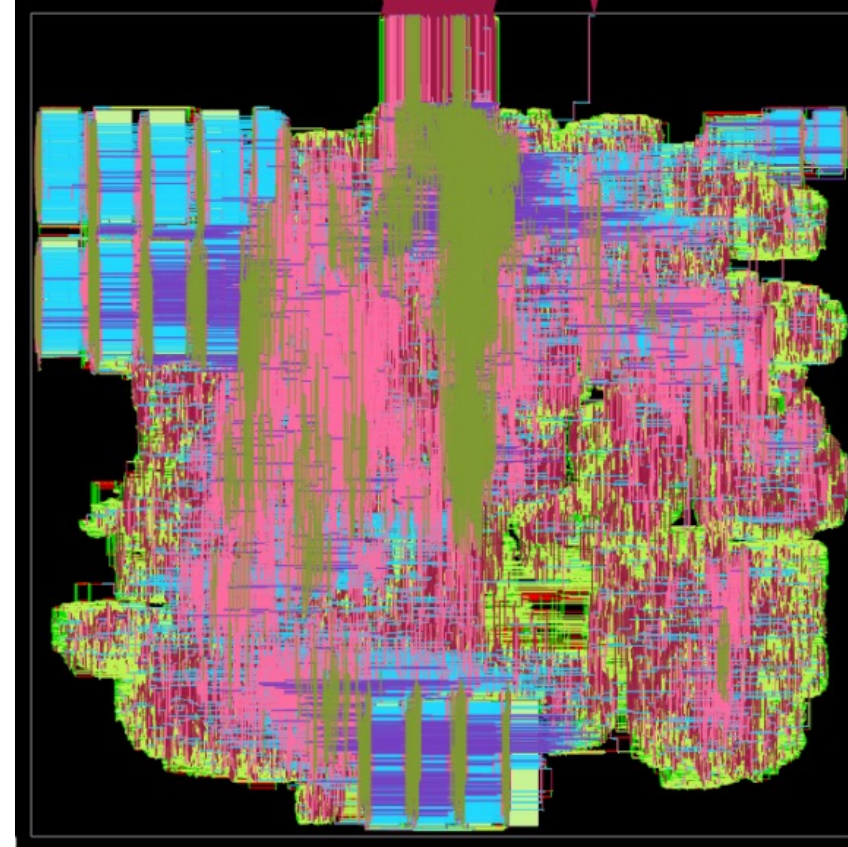
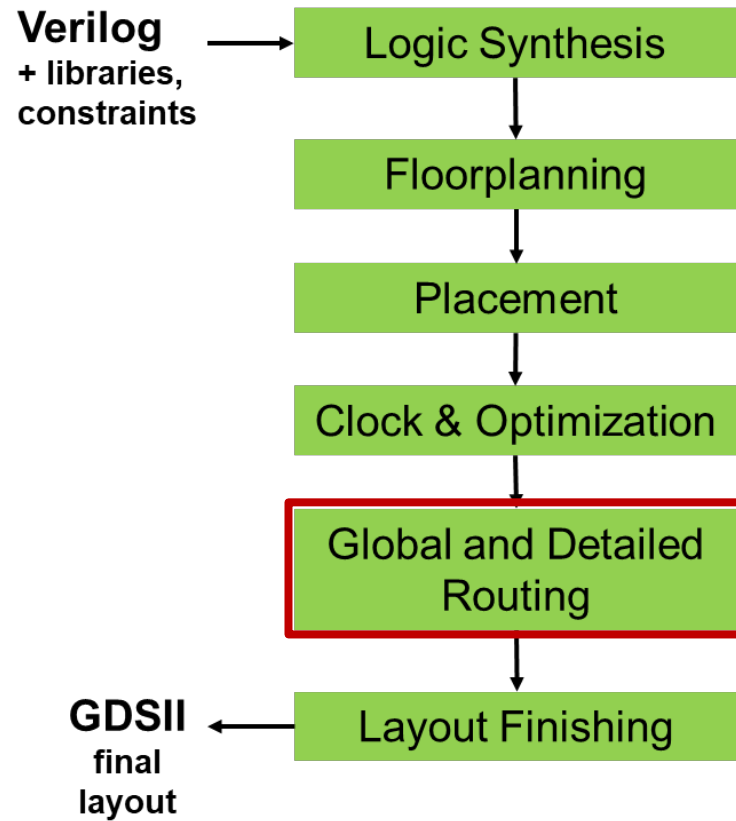
- Placement



- Routing

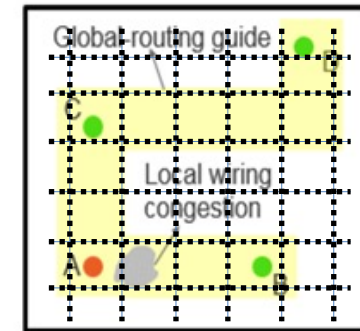


Final (Detailed) Routing

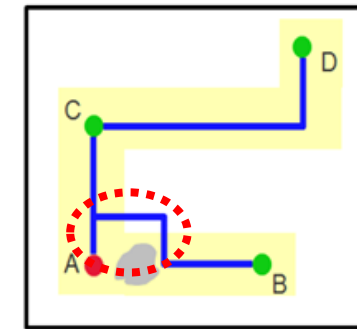


Background

- Routing challenges
 - Complex design rules
 - Enormous solution space
 - Physical and circuit considerations
- Generic “area routing” flow
 - Global routing
 - Produces 3D “route guides”
 - Detailed routing
 - Input: route guides = union of gcells
 - Output: physical nets
 - Subject to: honoring route guides, honoring design rules



GCell



Blockage-aware

<http://www.ispd.cc/contests/18/index.htm>

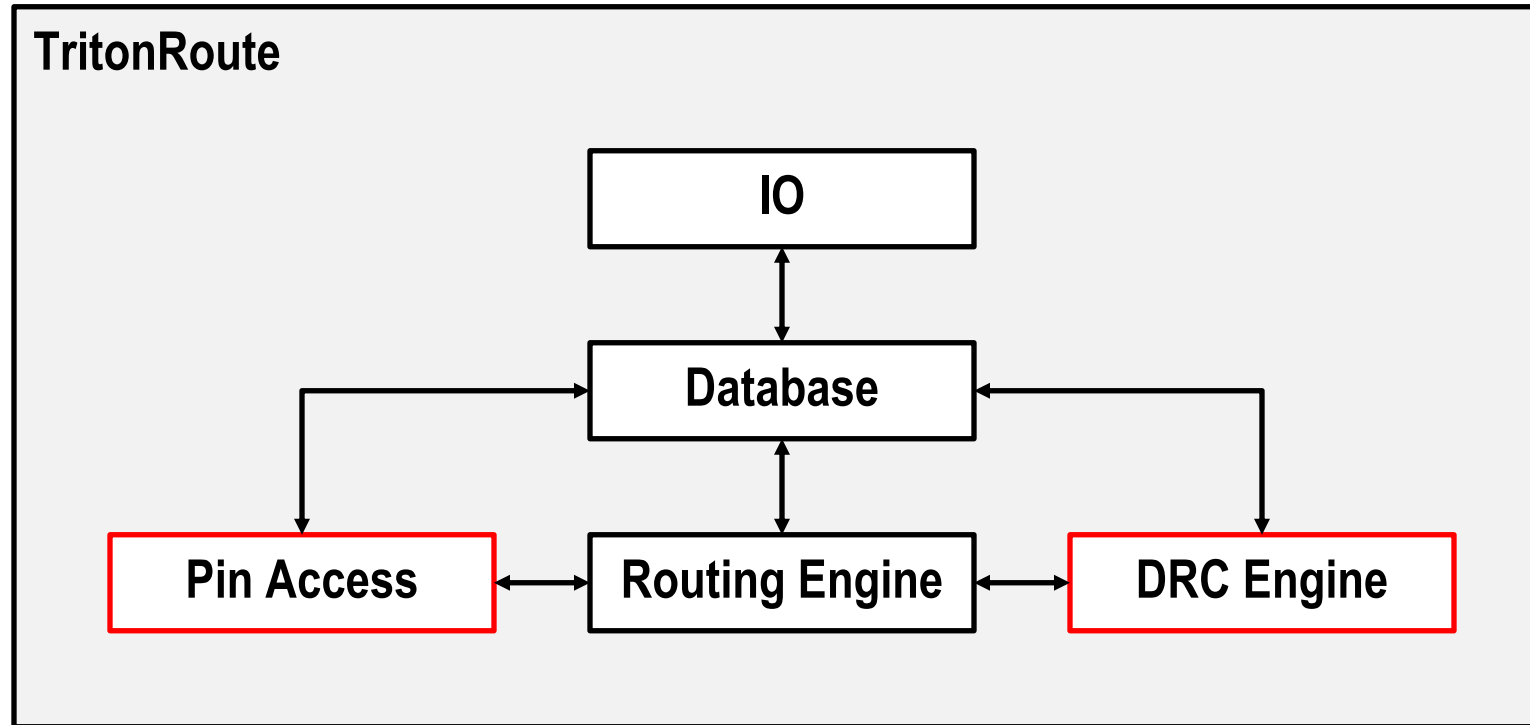
GCell = global routing grid;
Global router will only generate gcell-to-gcell connections

Critical Elements

- Must be able to clean up DRC (design rule check) violations !
 - Without a **DRC engine** → can't tell that violations exist !
 - Without violation filtering → have no clue what to ripup
- Major source of violations: naive **pin access**
 - On-track access assumption
 - No inter-cell pin access compatibility check
 - No accurate modeling of design rules
- **DRC engine and robust pin access are “scuba tanks”**
 - SCUBA: Self-Contained, Underwater, Breathing Apparatus

TritonRoute (2018-2022) – Overall Structure

- Best academic detailed router for contest benchmarks
- Only academic detailed router capable of delivering DRC-clean solution for commercial foundry nodes



Geometry-Based **Design Rule** **Checking** for Detailed Routing

Motivation

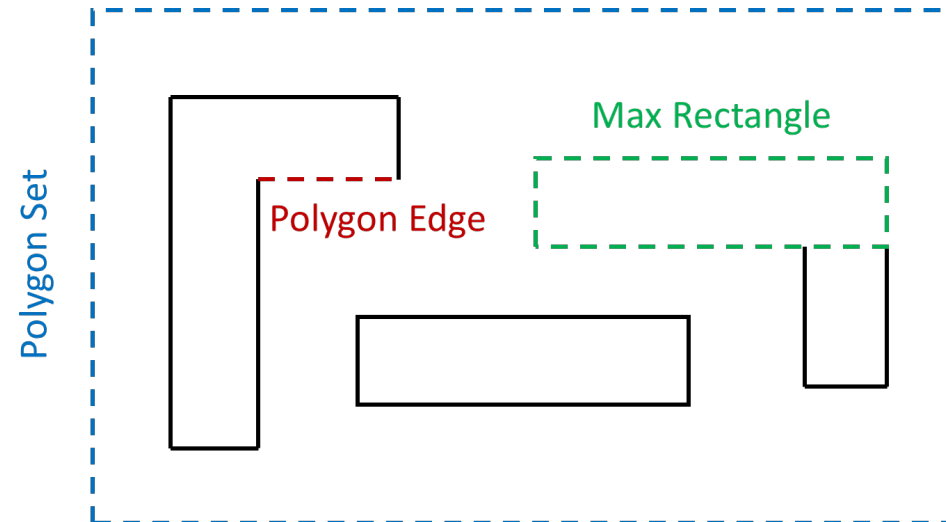
- Design rule checking is critical for EDA enablement
 - New technology has increasingly complex design rules
 - Mandatory physical verification for signoff
- **No** end-to-end framework for design rule checking in the open literature
 - → Missing key enablement for DRC convergence

Work at UCSD:

- (i) Optimized data structures for design rule check **for detailed routing**
- (ii) Industry-format (LEF) based design rule check methodology
- (iii) Differentiation between fixable and non-fixable design rule violations **for detailed routing**
- (iv) Foundry nodes: confirmed “clean” by commercial DRC tools

Preliminaries: Basic Geometry Objects

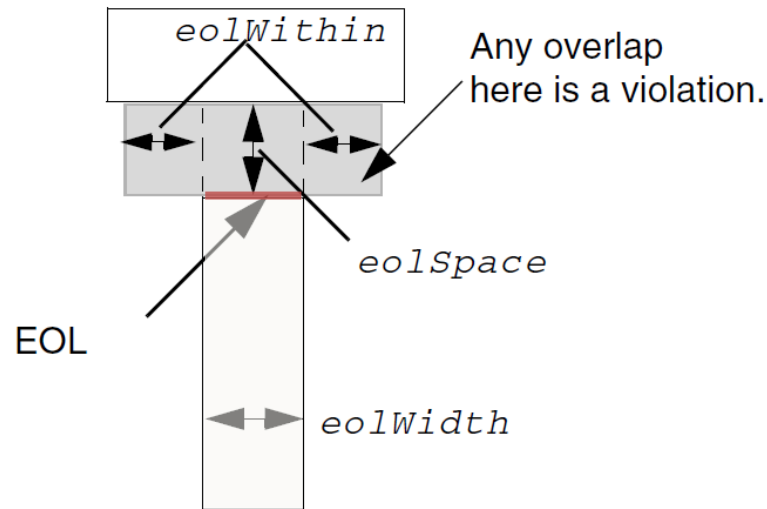
- Basic geometry objects in a DRC checking database
 - Polygon edge
 - = Edge of a polygon
 - Max rectangle
 - = Maximum rectangle inside a polygon
 - Polygon set
 - = Union of disjoint polygons



Preliminaries: Design Rule Syntax

- Typical design rule can have three components
 - Spacing value
 - Intrinsic property condition (optional trigger)
 - Extrinsic property condition (optional trigger)
- Example

SPACING eolSpacing ENDOFLINE eolWidth WITHIN eolWithin
Spacing value Intrinsic property Extrinsic property

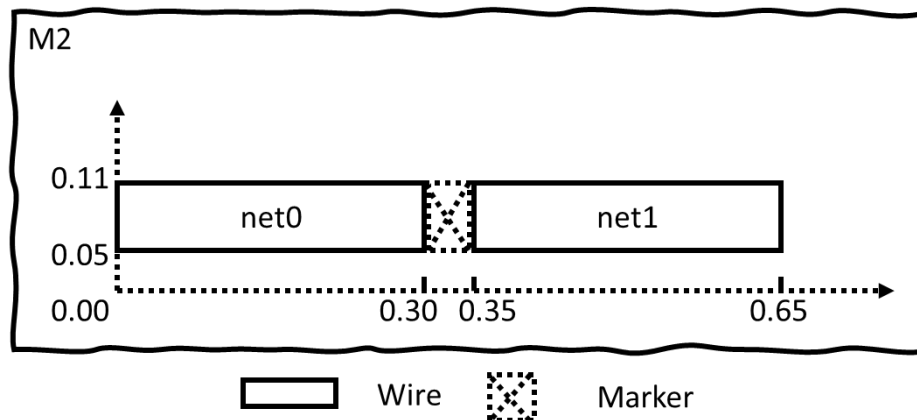


a) EOL width < *eolWidth* requires *eolSpace* beyond EOL to either side by < *eolWithin* distance.

EOL = END-OF-LINE

Preliminaries: Design Rule Violation Marker

- A design rule violation **marker** consists of
 - Bounding box
 - Layer
 - Violation net(s)
 - Design rule
- **Usage:** Give hints to DR where / what to ripup
- Example
 - Rule: SPACING 0.06
 - Object needs to be 0.06 unit away from each other

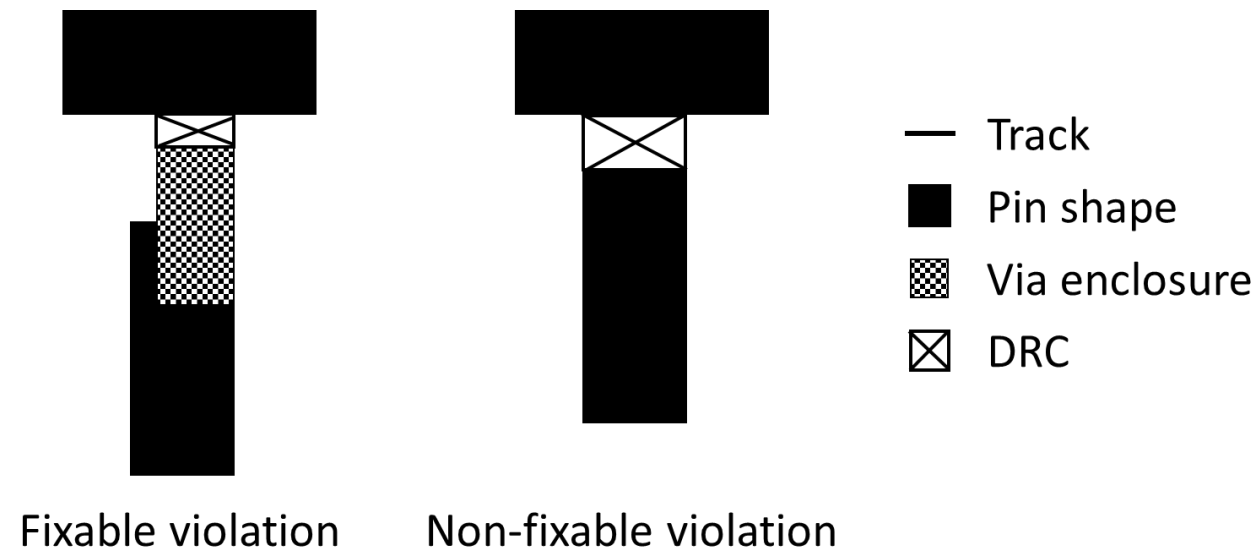


Marker

- Bbox: (0.30, 0.05) – (0.35, 0.11)
- Layer: M2
- Violation nets: net0 & net1
- Design rule: SPACING

Preliminaries: Fixable and Non-Fixable DRC

- Knowing locations of DRCs is not enough
 - DRC could happen inside standard cell itself
 - DRC could happen between PG stripes
 - DR cannot help resolve such **non-fixable** DRCs
- Need to filter DRCs before give them to DR
 - I.e., only provide DR with **fixable** DRCs
- Example



Problem Statement

- **Goal:** Given layout objects, find fixable design rule violations (if any)
- **Inputs**
 - Design layout database
 - Design rules
- **Constraints**
 - Design rule checking bounding box
 - Layer range (e.g., M2-M5)
- **Output**
 - Fixable design rule violation markers

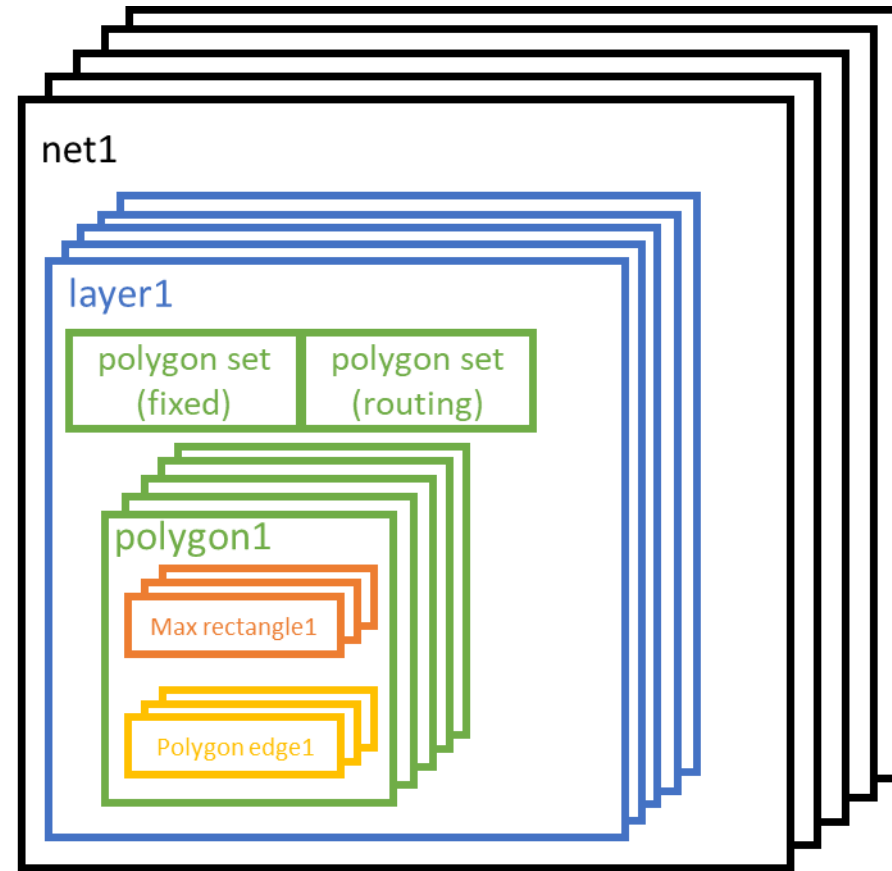
Database Objects

Object	Meaning	Status	Geometries
instTerm	cell pin	fixed	polygon(s)
term	block IO pin	fixed	polygon(s)
instBlockage	cell blockage	fixed	polygon(s)
blockage	block blockage	fixed	polygon(s)
pathSeg	regular net wire	routing	rectangle
pathSeg	special net wire	fixed	rectangle
via	regular net via	routing	rectangle(s)
via	special net via	fixed	rectangle(s)
patchMetal	regular net patch metal	routing	rectangle
patchMetal	special net patch metal	fixed	rectangle

Data Structure

- Object (fixed) & object (routing)
→ Key for fixable / non-fixable differentiation
- Polygon set (fixed)
 - Union of fixed obj. shapes
- Polygon set (routing)
 - Union of routing obj. shapes

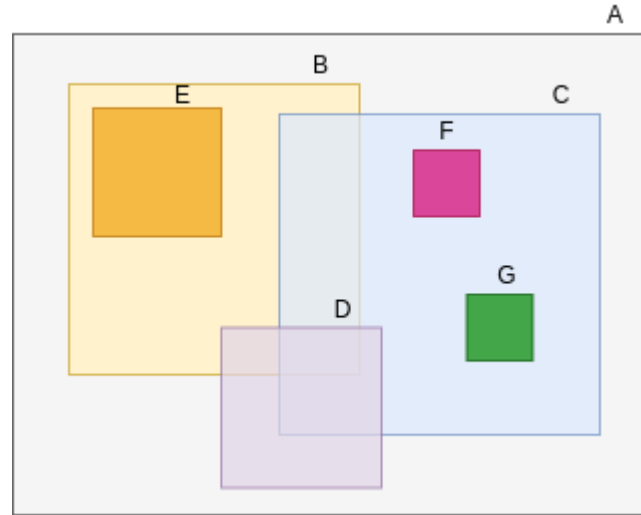
Object	Status
instTerm	fixed
term	fixed
instBlockage	fixed
blockage	fixed
pathSeg	routing
pathSeg	fixed
via	routing
via	fixed
patchMetal	routing
patchMetal	fixed



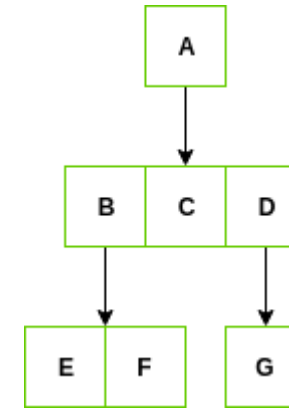
DRC checking database

Region Query

- RTree as underlying container



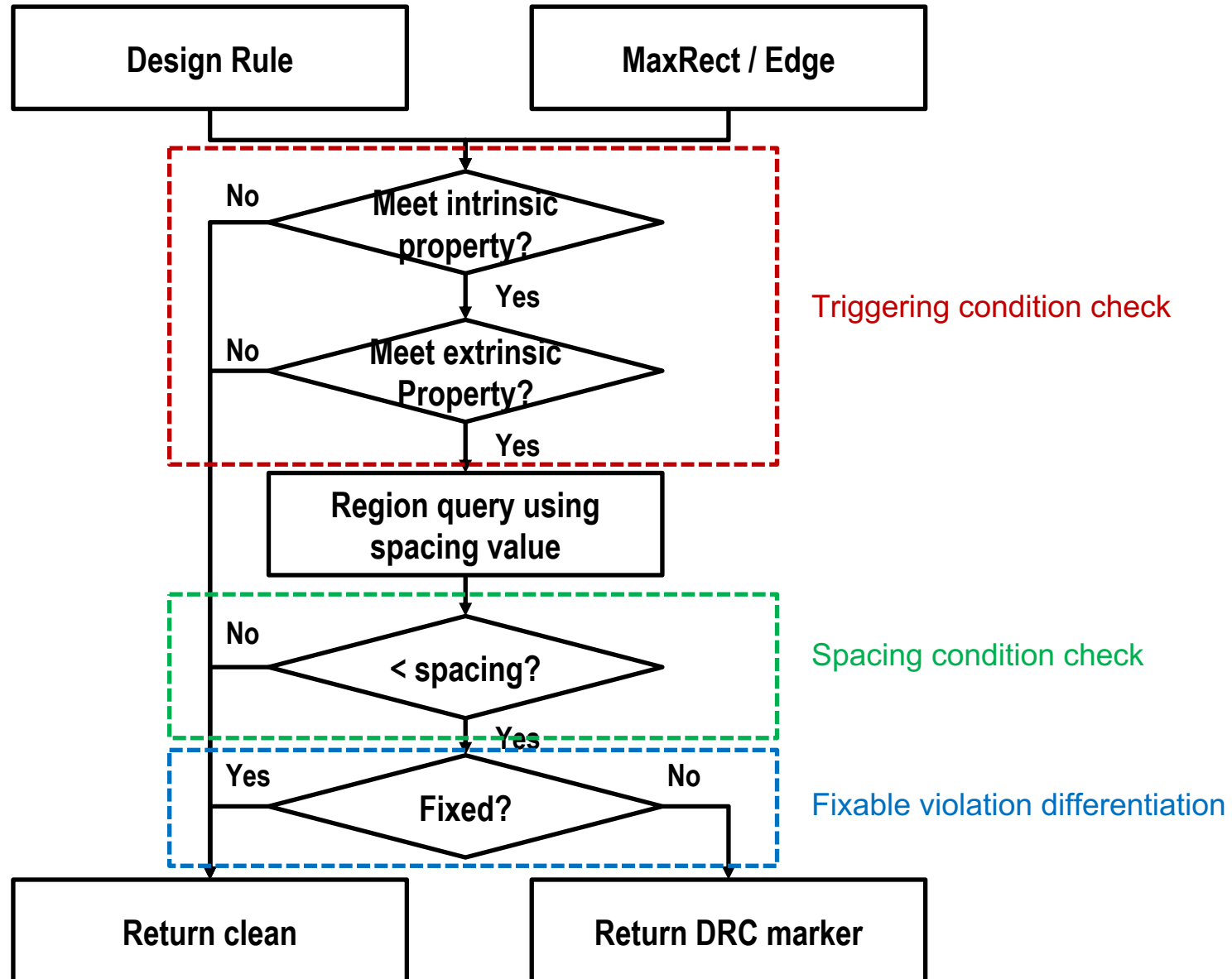
Layout



RTree Representation

- Two RTrees for each layer
 - MaxRect RTree
 - Edge Rtree
- Each maxRect / Edge has property indicating whether it is **fixed** or **routing**

High-Level Flow for DRC Check (Single Obj.)



Design Rules Types

- Design rules are divided into two categories
→ **Metal** layer and **Cut** layer

- **Metal layer rules**

- Metal short
- Non-sufficient-metal-overlap
- Parallel run length (PRL) spacing
- Minimum width
- Minimum step
- End-of-line (EOL) spacing

} Metal Shape Rule

} Metal Spacing Rule

- **Cut layer rules**

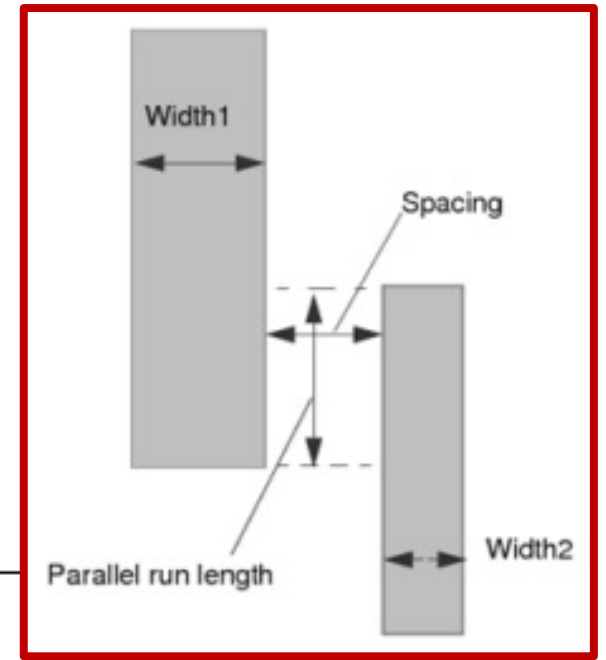
- Cut short
- Cut spacing

Many more design rule types in advanced technologies! See the “LEF5.8” standard, e.g., [here](#).

Metal Spacing Checking

Algorithm 1 Check metal spacing

```
1: Input: max rectangle  $m$ 
2:  $N \leftarrow \text{queryMaxRectangles}(m, \text{maxDist})$ 
3: for all  $n \neq m$  in  $N$  do
4:   if  $\text{isOverlap}(m, n)$  then
5:     if  $\text{getNet}(m) = \text{getNet}(n)$  then
6:        $\text{checkNSMetal}(m, n)$ 
7:     else
8:        $\text{checkMetalShort}(m, n)$ 
9:     end if
10:  else
11:     $\text{checkPRL}(m, n)$ 
12:  end if
13: end for
```



Algorithm 3 Check non-sufficient metal overlap

```
1: Input: max rectangles  $m, n$   
2:  $nsRect \leftarrow \text{getIntersection}(m, n)$   
3: if  $\text{diagLen}(nsRect) \geq \text{minWidth}$  then  
4:   return  
5: end if  
6: if  $\text{width}(m) < \text{minWidth}$  OR  $\text{width}(n) < \text{minWidth}$  then  
7:   return  
8: end if  
9: if  $\text{hasValid3rdObj}(nsRect)$  then  
10:  return  
11: end if  
12:  $\text{addMarker}(\text{NonSufficientMetalOverlap})$ 
```

Min Width Checking

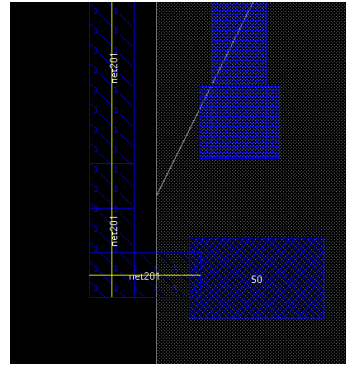
Algorithm 5 Check minimum width

```
1: Input: polygon  $m$ 
2:  $N \leftarrow \text{slicePolygon}(m, \text{vertical})$ 
3: for all  $n$  in  $N$  do
4:   if  $y\text{Span}(n) \geq \text{minWidth}$  then
5:     return
6:   end if
7:   if not  $\text{hasRouting}(n)$  then
8:     return
9:   end if
10:   $\text{addMarker}(\text{MinimumWidth})$ 
11: end for
12:  $N \leftarrow \text{slicePolygon}(m, \text{horizontal})$ 
13: for all  $n$  in  $N$  do
14:   if  $x\text{Span}(n) \geq \text{minWidth}$  then
15:     return
16:   end if
17:   if not  $\text{hasRouting}(n)$  then
18:     return
19:   end if
20:   $\text{addMarker}(\text{MinimumWidth})$ 
21: end for
```

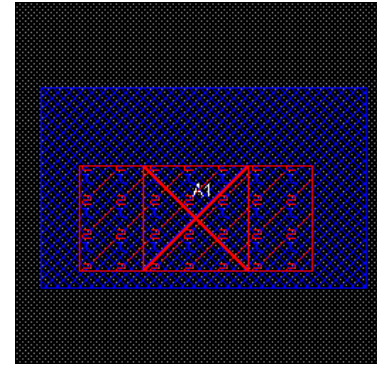
Dynamic Programming-Based Multi-Level **Pin Access Analysis**

Motivation

- Pin access = wire / via connection to access a pin



Wire access



Via access

- Critical to decrease DRCs in detailed routing
→ Failure results in repeating violation patterns
- **Need robust & scalable pin access analysis (!)**

Previous Works / Our Work

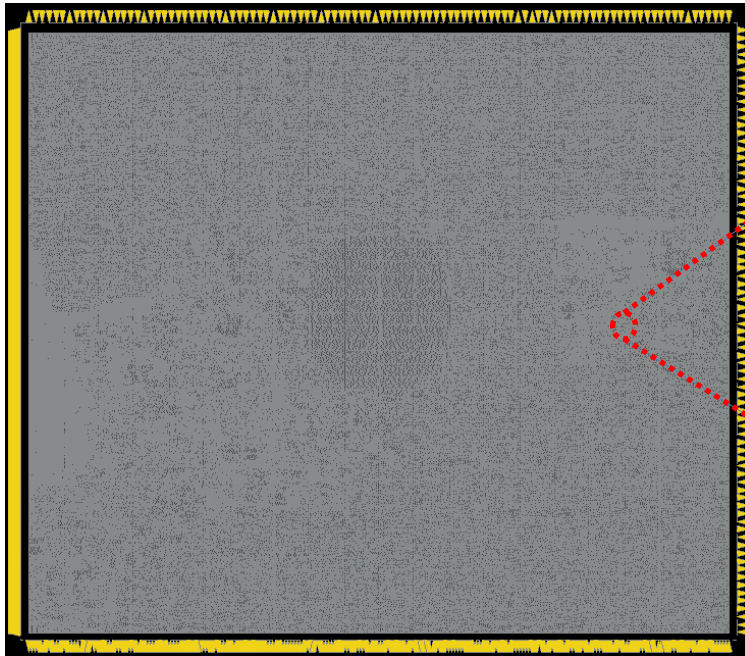
- Existing work [Han15] assumes **on-track** access
- Usually assume alignment between routing track and placement site
 - Not always true (ISPD18/19 contests)
- LUT-based abutting cell pair analysis [Xu16]
 - Not scalable ($> 10M$ combinations)

Our work:

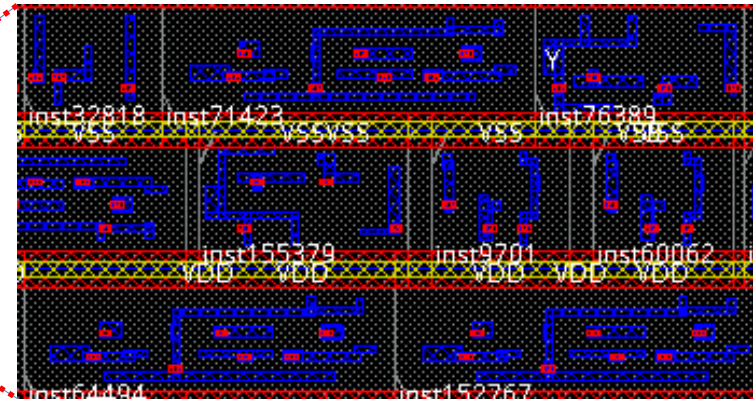
- (i) Robust pin access point enumeration
- (ii) Boundary conflict-aware access pattern enumeration
- (iii) Dynamic programming-based access pattern selection for standard cell instance cluster

What Does Pin Access Analysis Do?

- Testcase: ISPD18_test10
 - 290K standard cell instances
 - 992K nets
- DRC clean pin access pattern selection in 241s



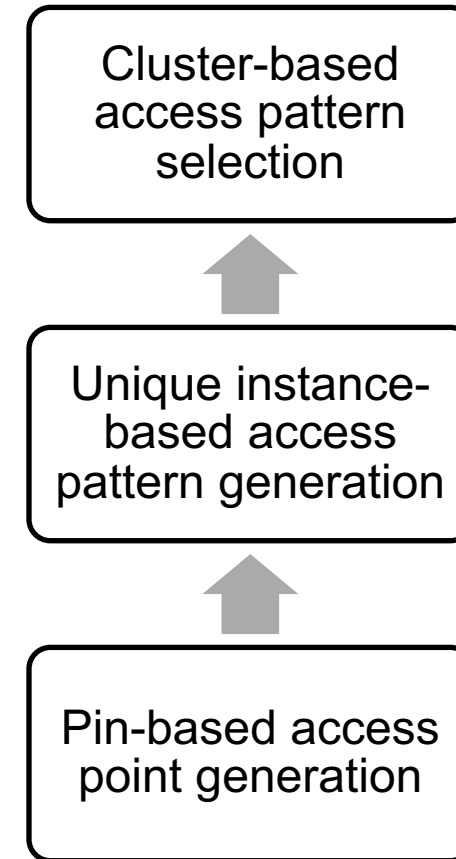
Whole design layout



Zoomed-in region
(red dots are via access locations)

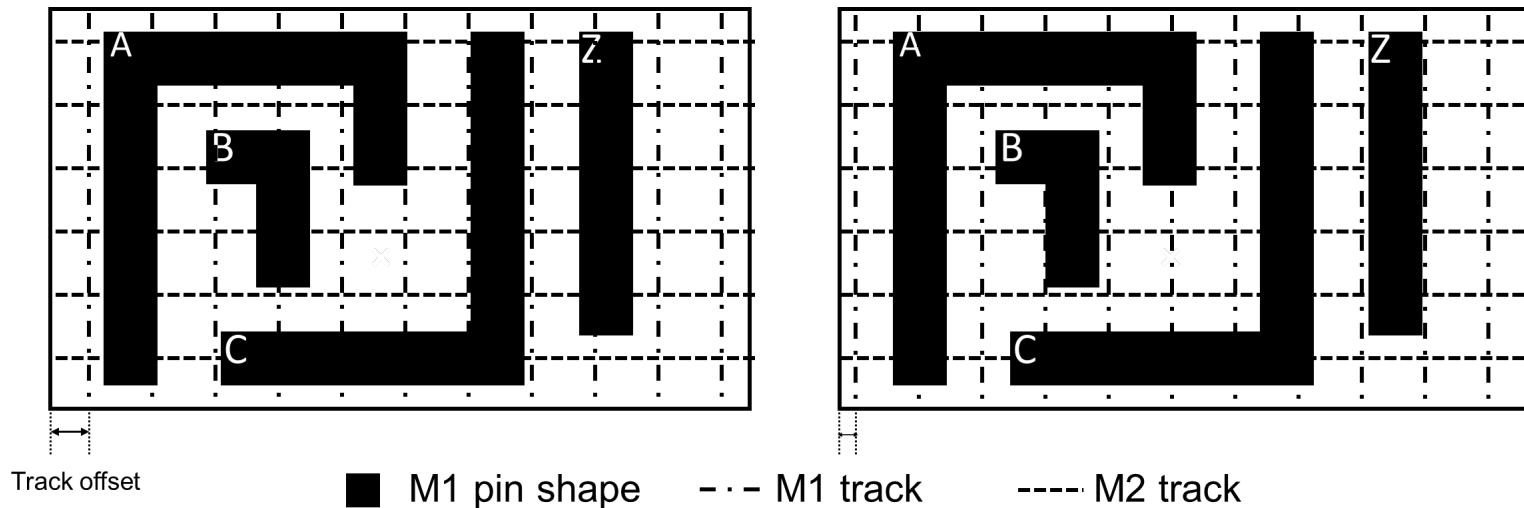
How to Find Such Access Points?

- **Multi-level hierarchical pin access analysis**
 - Unique instance pin level
 - Unique instance level
 - Instance cluster level
- **Scalable memory usage**
 - On-demand design-based analysis
- **Scalable runtime**
 - DRC check on the fly
 - More than 2M DRC engine calls in 8min with single thread



Unique Instance

- To address track-placement site misalignment
- Defined by a **signature** consisting of
 - Cell master (e.g., BUFFX4)
 - Orientation
 - Offsets to all track patterns
- Two cell instances point to the same unique instance if they share the same **signature**

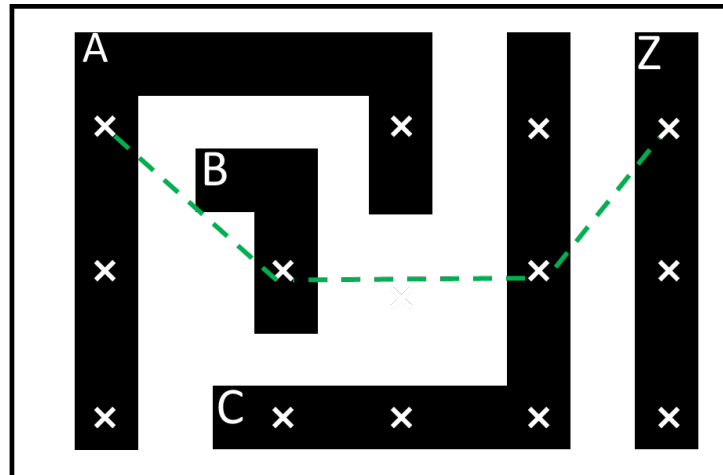


Two different unique instances due to M1 track offset

Thanks: Dr. Bangqi Xu

Definitions

- **Access point** (for a pin)
 - A location (x, y, layer) that detailed router (DR) can make route to
- **Valid access point**
 - An access point that allows DRC-clean routing
- **Valid access pattern:** combination of mutually DRC-clean access points (one access point per pin)



■ Pin shape × Access point - - - Access pattern

Coordinate Types

1. **On-track:** On preferred and non-preferred routing track of upper layer
2. **Half-track:** At midpoint between two neighboring routing tracks
3. **Shape-center:** At midpoint between left and right (or top and bottom) coordinates of a rectangular pin shape
4. **Enclosure boundary:** Via enclosure aligns with pin shape boundary

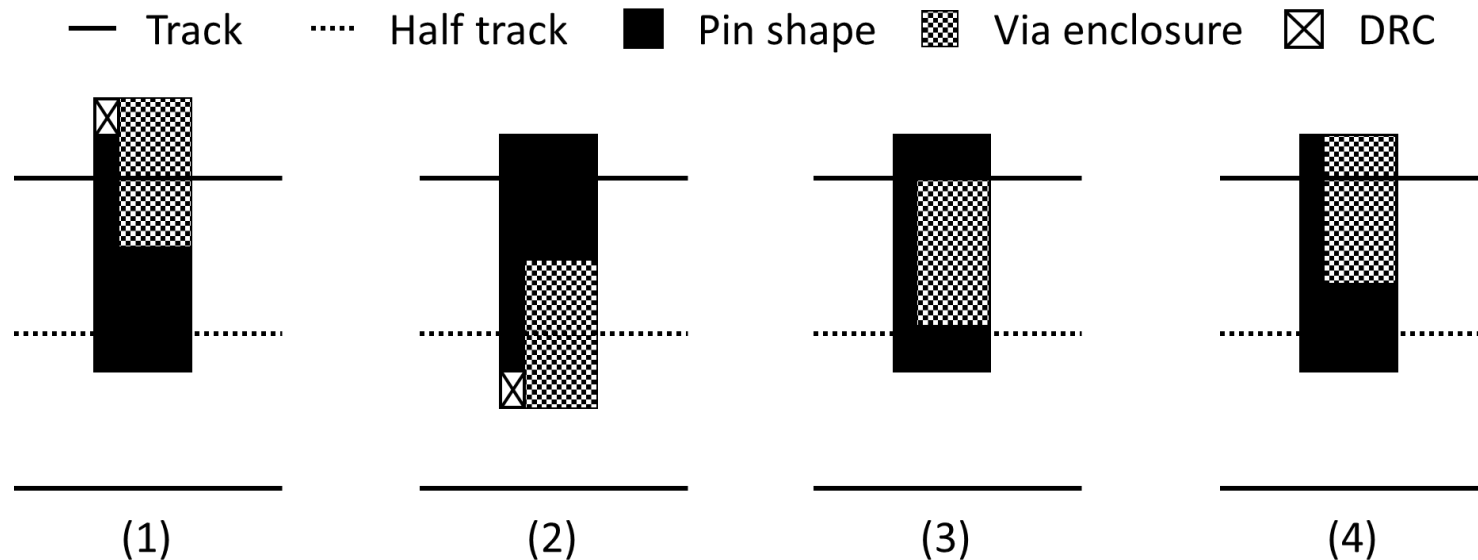


Illustration of y-coordinate types

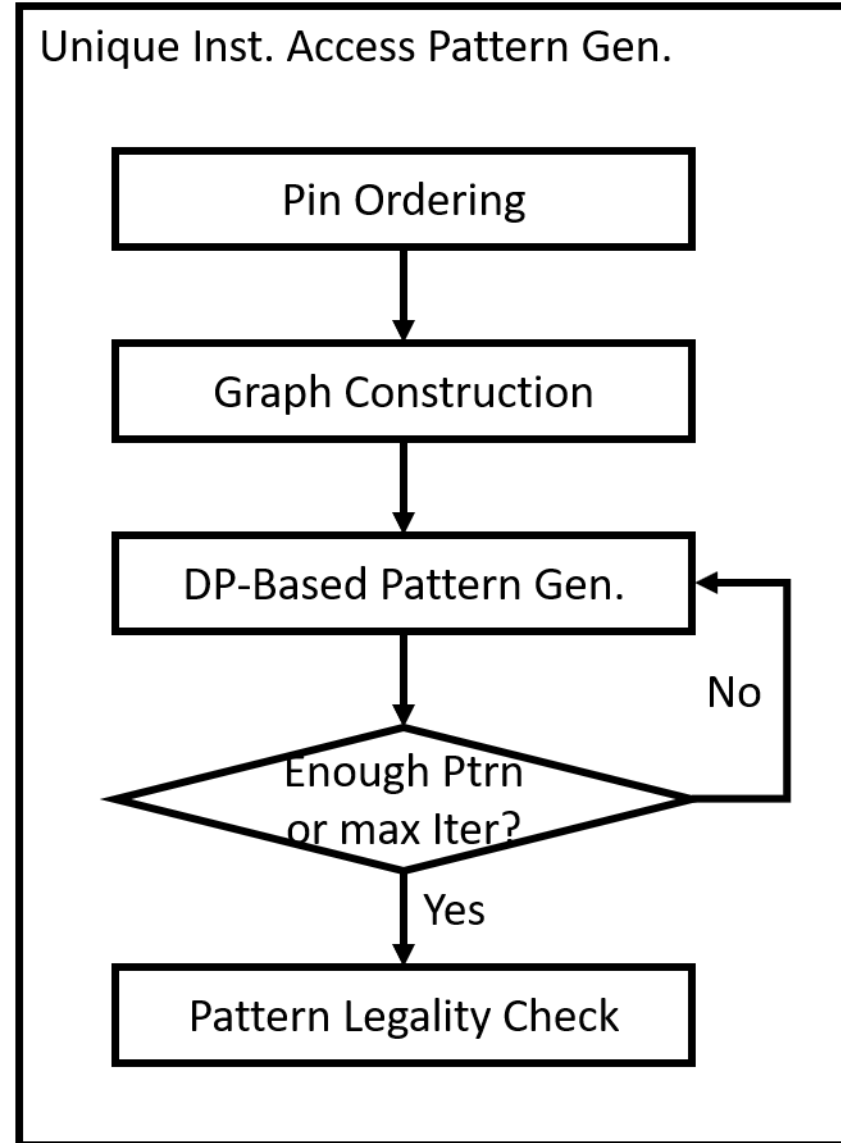
Pin-Based Access Point Generation

Algorithm 1 Pin-based access point generation

```
1: Inputs:  $pin$ , track patterns  $tps$ , viadefs  $vias$ 
2: Output: valid access points  $aps$ 
3: for all nonPreferredDirCoordType  $t1 \in \{0, 1, 2\}$  do
4:   for all preferredDirCoordType  $t0 \in \{0, 1, 2, 3\}$  do
5:      $tmpAps \leftarrow \text{genAccessPoint}(pin, tps, vias, t0, t1)$ 
6:     for all  $ap \in tmpAps$  do
7:       if isValid( $ap$ ) then
8:          $aps += ap$ 
9:       end if
10:    end for
11:    if  $|aps| \geq k$  then
12:      return
13:    end if
14:  end for
15: end for
```

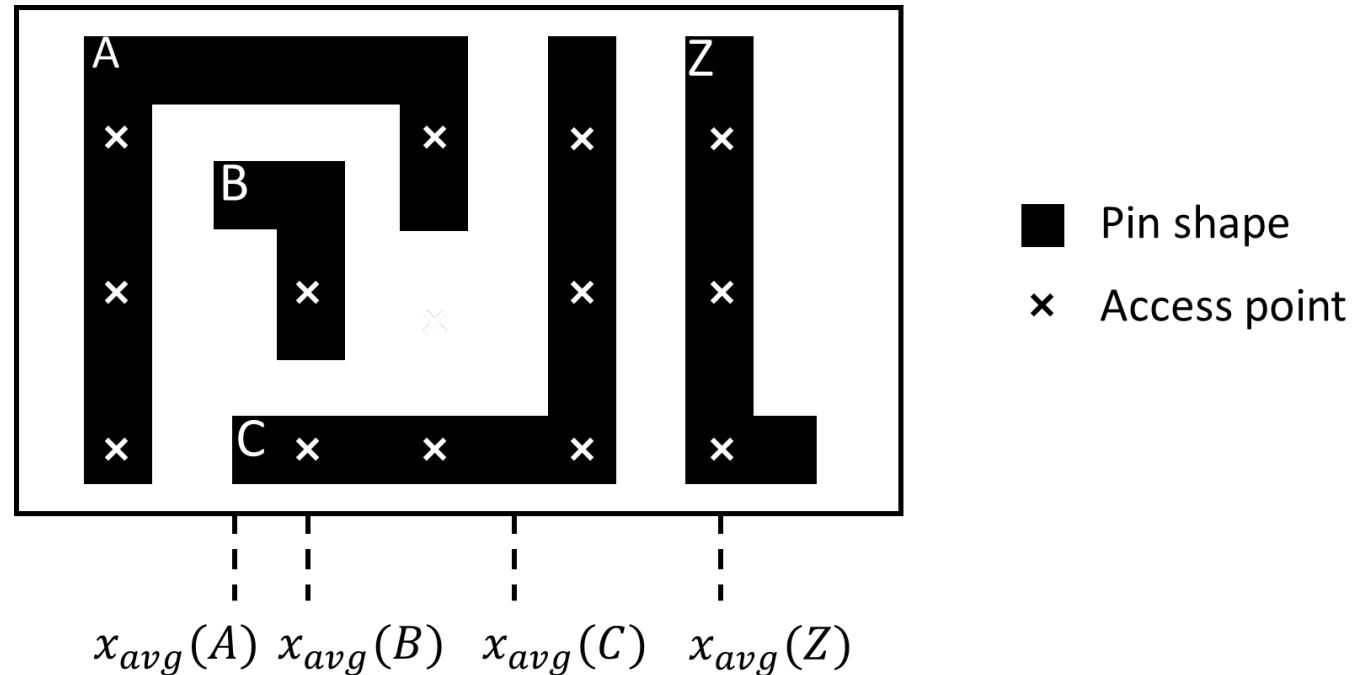
Unique Instance-Based Access Pattern Generation

- Input
 - Valid access points of pins in a unique instance
- Output
 - Valid access patterns



Pin Ordering

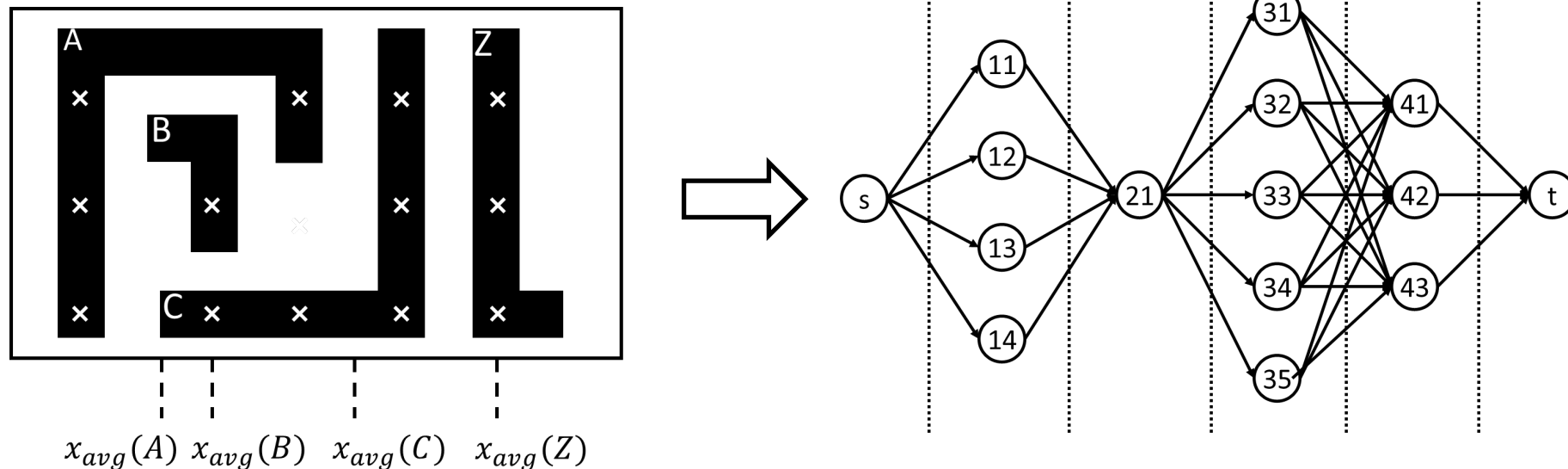
- Sort pins according to their average x coordinate of valid access points



- Idea: neighbors in sorted pin list are more likely to have conflicts due to DRC
 - I.e., pin A and pin B are more likely to have conflict compared to pin A and pin C

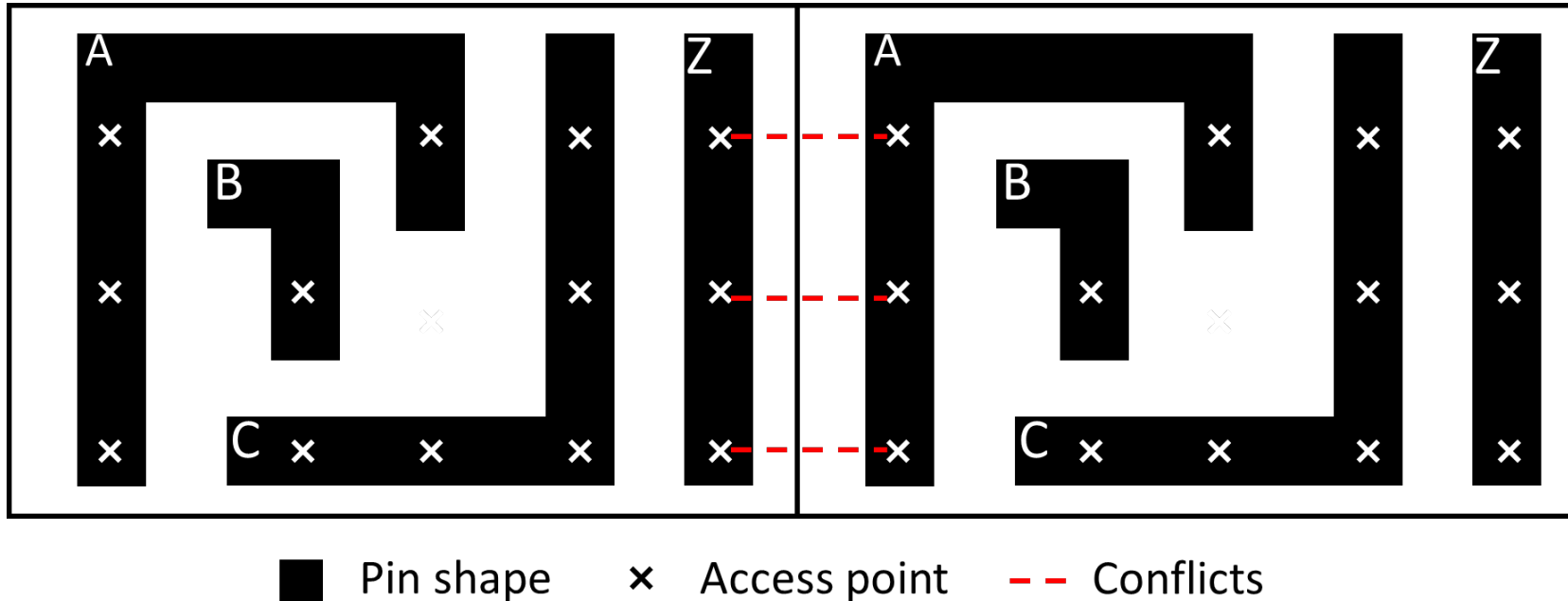
Graph Construction

- Vertex = access point
 - Marked with pin index and access point index
 - E.g., 23 means the third access point of the second pin
 - s and t are virtual start and end points
- Pin correspond to a “group” of vertices in graph
- Edge exists between pair of access points from neighboring groups, weighted by physical distance
- Access pattern = path from s to t



Iterative Edge Penalty Method

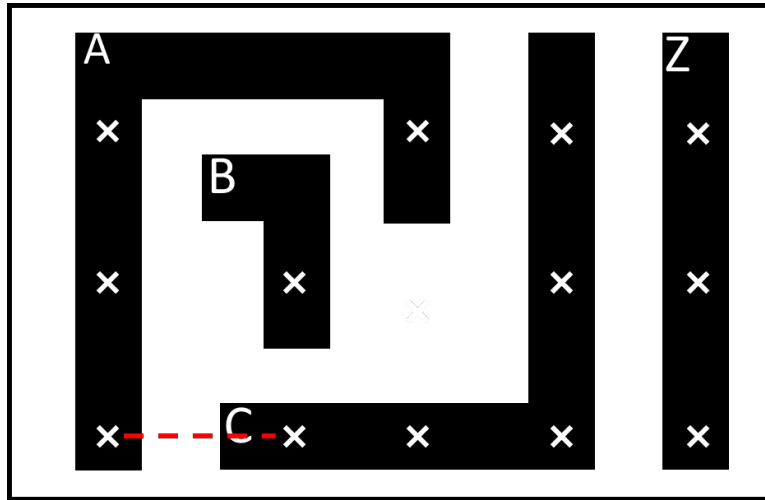
- Inter-cell pattern conflicts between cell-boundary pins
→ Need to encourage to choose different boundary pin access point



- **Solution: add penalty cost to boundary pin access points if they have been selected in existing pattern**

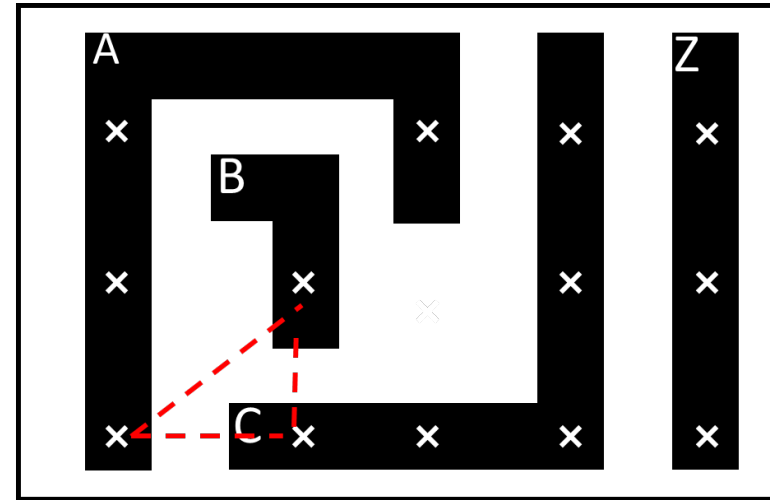
Pattern Legality Check

- Use DRC check engine to validate access pattern
 - i. Violation can occur between non-neighboring pins
 - ii. Some design rules check multiple objects (access points)



■ Pin shape × Access point - - Conflicts

(i)



(ii)

- Only **DRC-clean** patterns will be seen in next stage

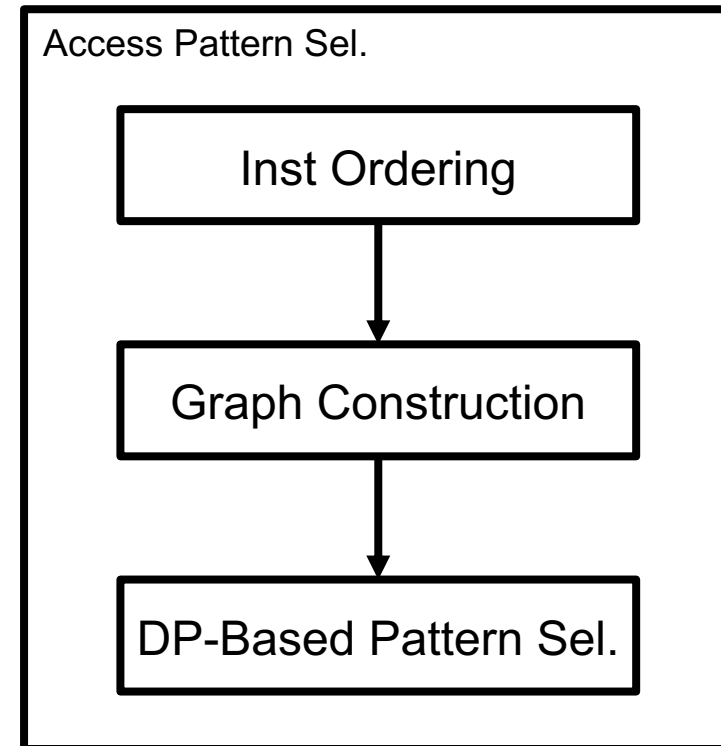
Cluster-Based Access Pattern Selection

- **Inputs**

- Instances in a cluster (of the same row)
- Access patterns of each unique instance
- Map from instance to corresponding unique instance

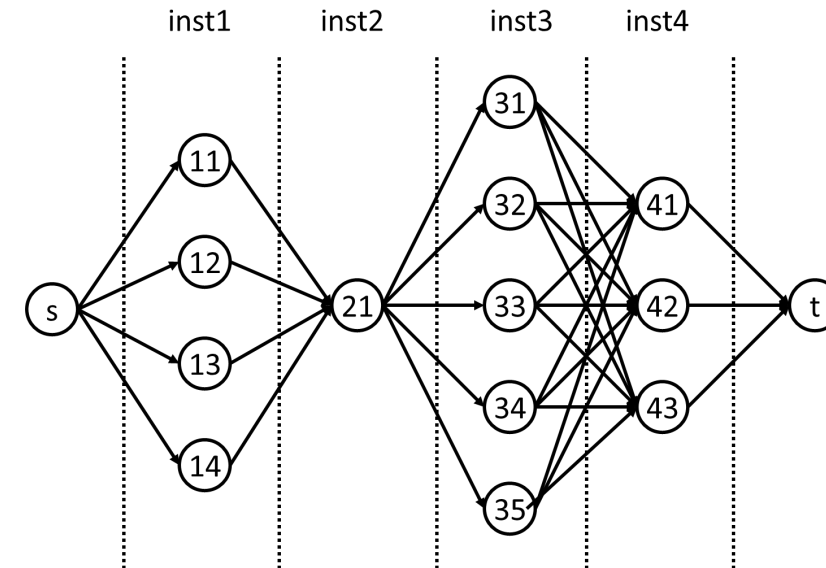
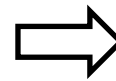
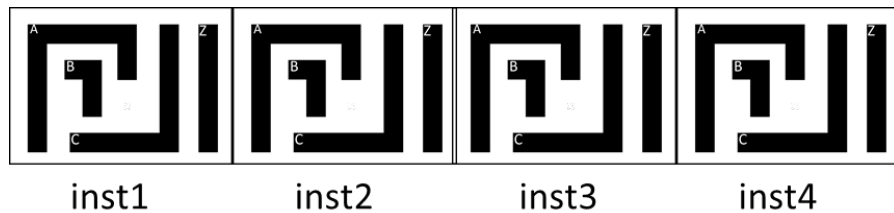
- **Output**

- Access pattern for each instance in the cluster with minimized overall cost



Cluster-Based Access Pattern Selection

- Instance ordering
 - Sort instances in the cluster according to x coordinate of the lower-left corner
- Graph construction
 - Vertex = access pattern
 - Shortest path from s to t is the best pattern combination



- DP formulation
 - Similar as previous formulation
 - No iteration

Also: Routing to IOs: pad (ICeWall)

SOC Integration and Planning: ICeWall Pading Gen

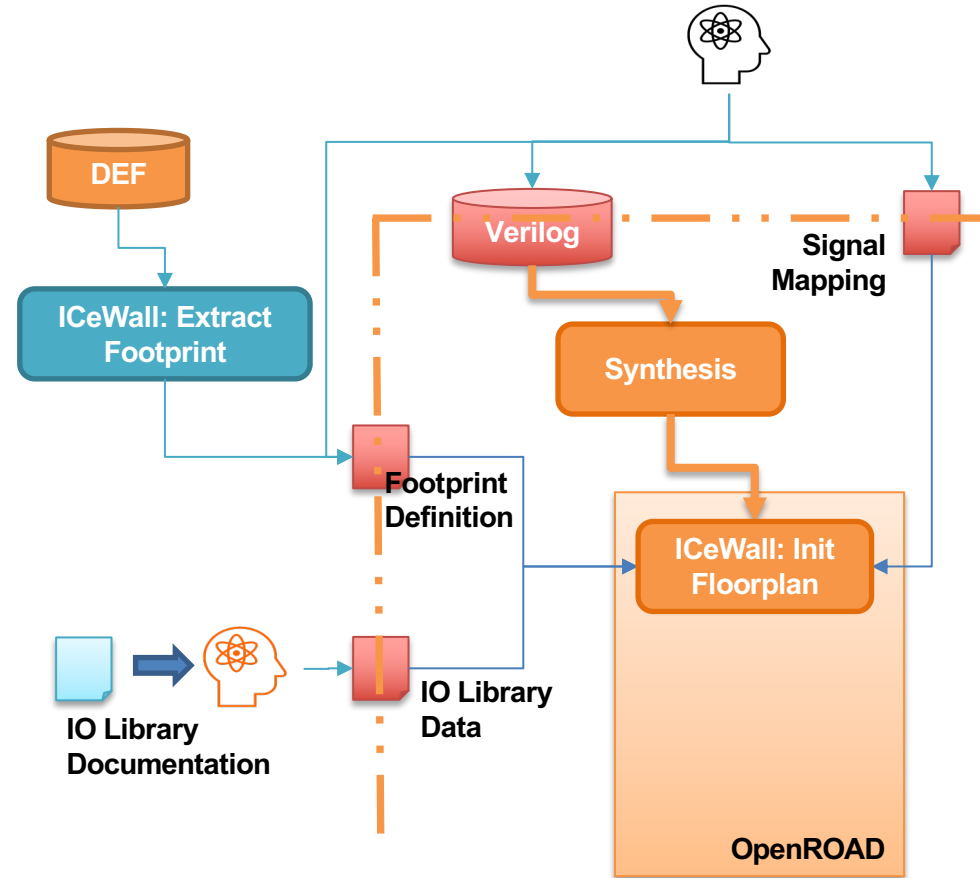
- **Starts with:**

- Verilog netlist with signal IO pads for simulation and STA
- Power/ground IO cells may be present
- IO cell data (signal, P/G, fillers, ...) from library documentation

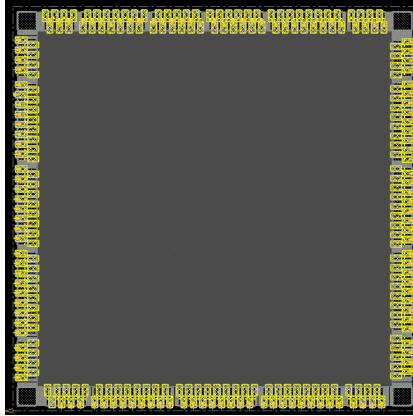
- **Footprint file** defines where each padcell is to be placed in the padding – supports reuse of pre-existing padframes

- **Signal mapping file** defines which signal in the Verilog is to be associated with which padcell in the padding
 - + Auto-assignment capability in ICeWall

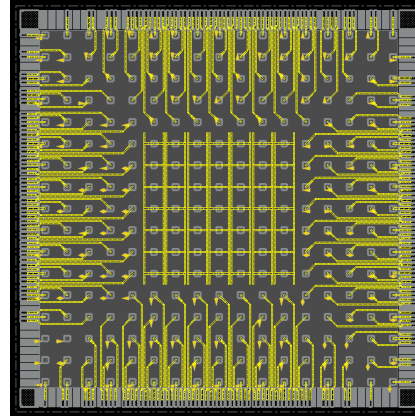
- **Decouples footprint and signal mapping** for padframe reuse



ICeWall Padding Examples

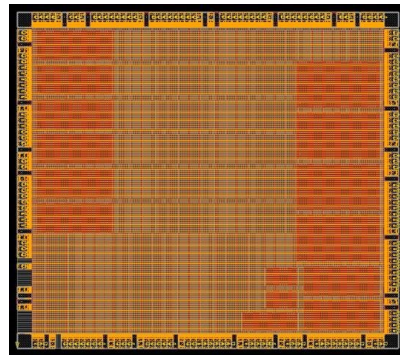


GF12LP BP-1,
staggered pads



GF12LP BP-1,
as a flipchip

SKY130
coyote, + pads



What designers ask for ...



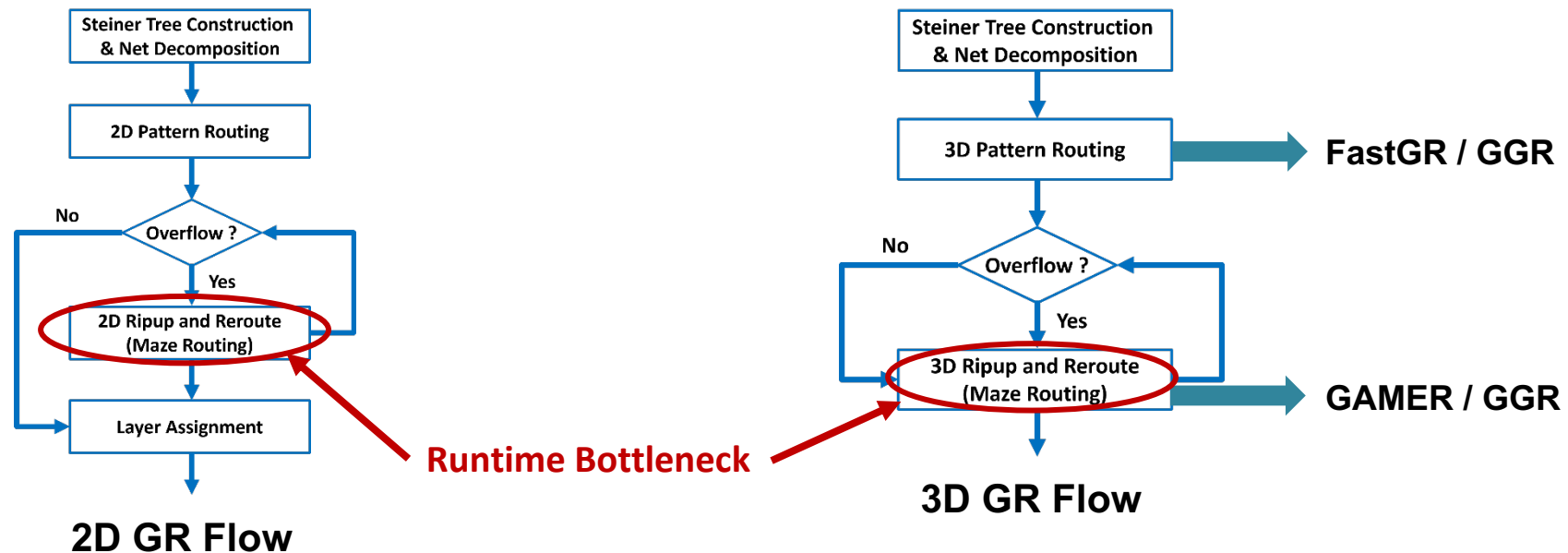
- Determining the number of required P/G pads to be provided as callback functions to allow to encapsulate specs from library documentation
- Definition of padding segments for analog signals, PHYs, different IO voltages, etc.
- Definition of control cells that are required on a per-IO cell basis

Also: GPU-Accelerated GRT

Summary of Previous Methods

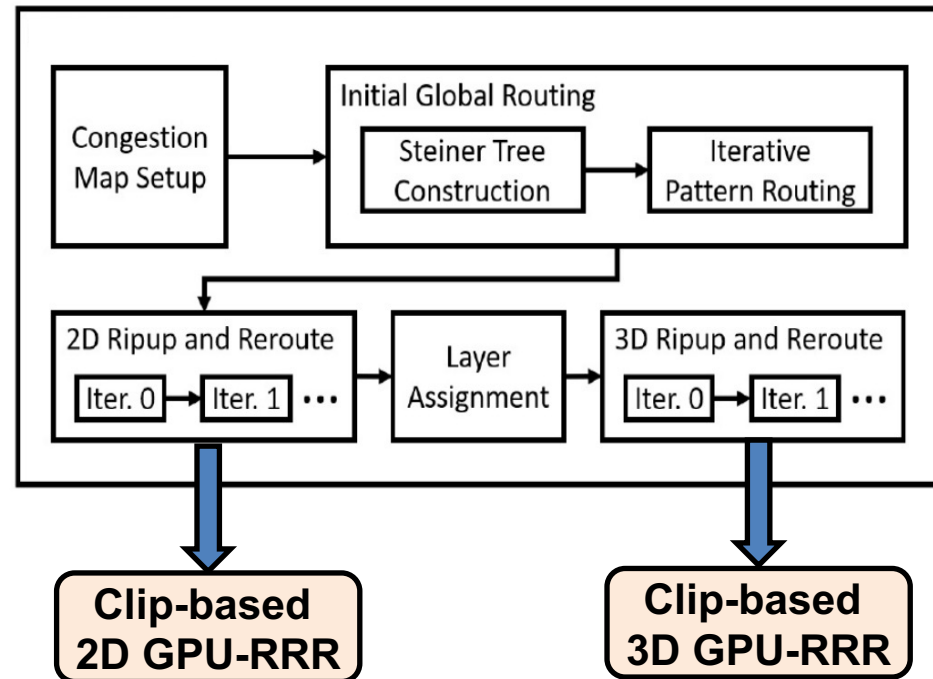
- 2D-GPU-accelerated GRs are based on FastRoute4.1
 - **SPRoute** and **SPRoute2.0**: implement parallel maze routing
- 3D-GPU-accelerated GRs are based on CUGR
 - **FastGR** and **GGR**: implement parallel L/Z-shape pattern routing
 - **GAMER** and **GGR**: implement parallel maze routing
 - Replace the A* search algorithm with the parallel n-bend pattern routing algorithm

Route a batch of non-overlapping nets concurrently !

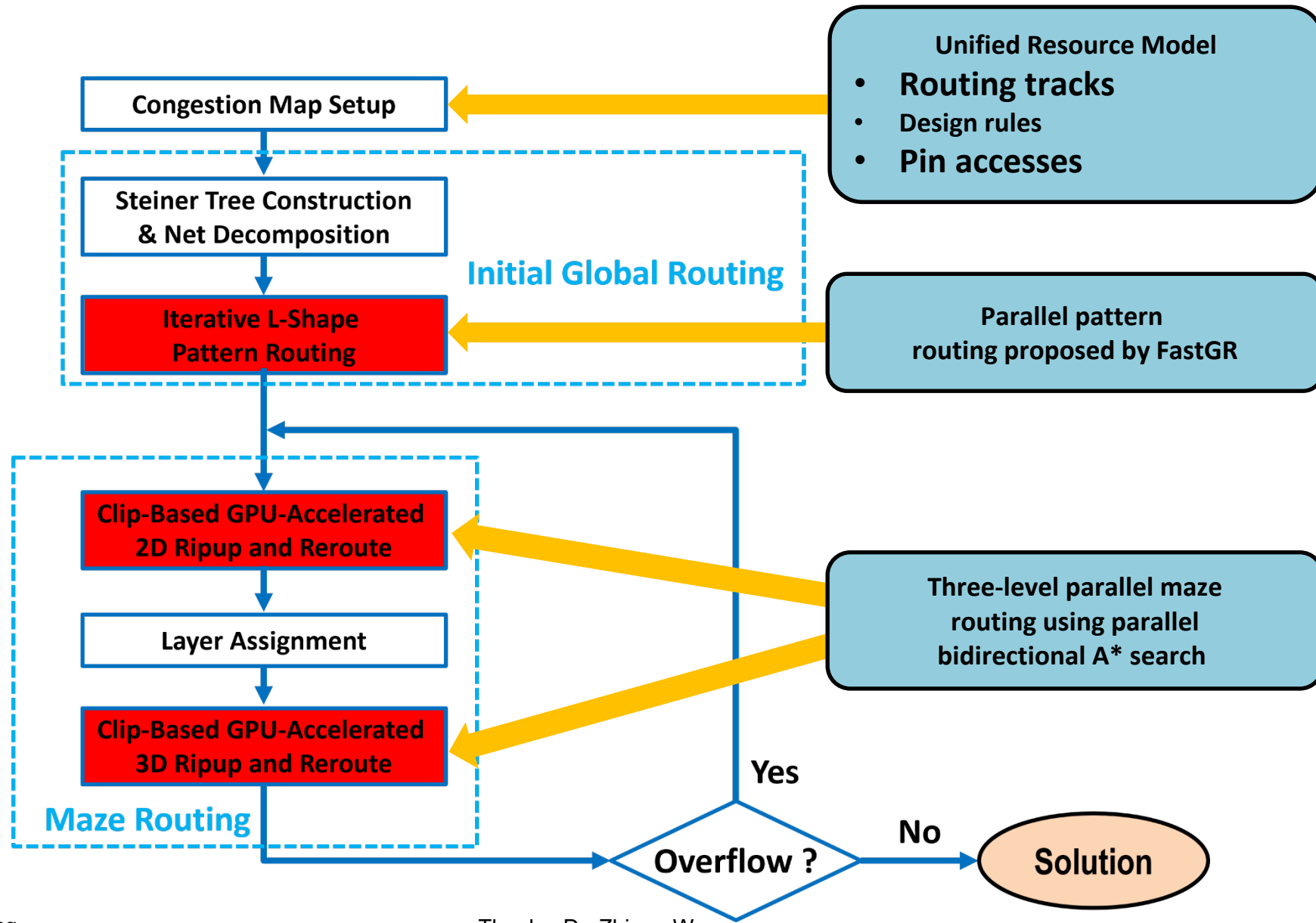


Proposed GPU-Accelerated TritonRoute-GR

- Our GPU-GR is based on TritonRoute-GR
 - TritonRoute-GR adopts a two-step approach (2D + 3D GR)
 - 2D global routing can effectively reduce the solution space
 - 3D global routing can further optimize the solution locally
 - Replace the original 2D-RRR and 3D-RRR with corresponding GPU-accelerated GPU-RRR

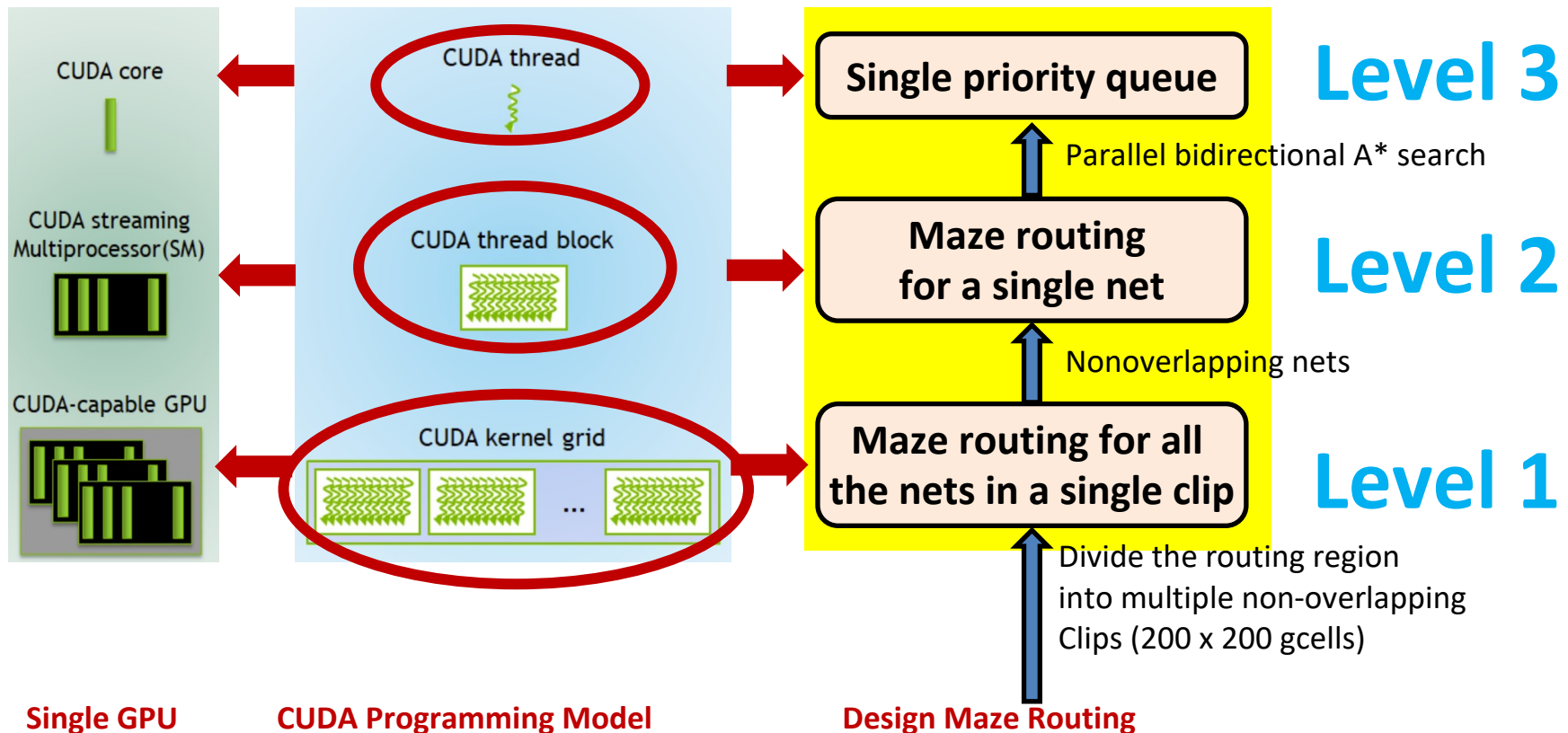


Current Approach



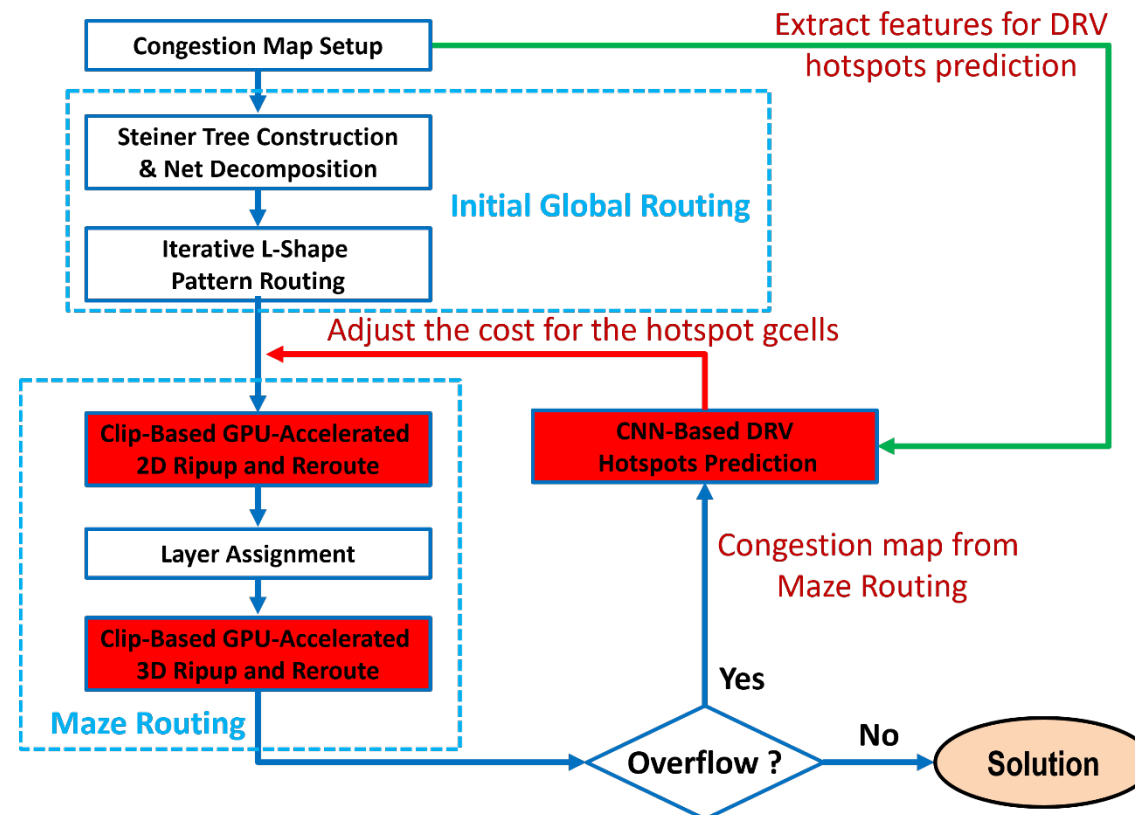
Three-Level Parallel Maze Routing

- We adopt three-level parallel maze routing
 - Level 1: maze routing for all the nets in a single clip $\leftrightarrow$ Grid on GPU
 - Level 2: maze routing for a single net $\leftrightarrow$ Block on GPU
 - Parallel A* search based on multiple priority queues [[ZhouZ'15AAAI](#)]
 - Level 3: expansion for a single priority queue $\leftrightarrow$ Thread on GPU



Future Work

- “machine learning alongside optimization algorithms”
- Combine the detailed-routability-driven GR with the ML-based DRV prediction models
- Improve accuracy and robustness compared to “end to end learning”



Thanks: Dr. Zhiang Wang

BACKUP
